

INF 662 · DESARROLLO DE APLICACIONES MÓVILES

Guía de Laboratorio N.º 3

Anatomía de un proyecto Flutter y tu primera interfaz con widgets

MATERIA INF 662 — Desarrollo de Aplicaciones Móviles**SEMESTRE** 6.º · Mención Ing. de Software**TEMA** Estructura del proyecto y widgets**DURACIÓN** 5 horas (1 sesión de laboratorio)**CAPÍTULO** Cap. 2 — Explorando Widgets y navegación**MODALIDAD** Laboratorio · individual**DOCENTE** M. Sc. Lic. Huáscar Fedor Gonzales Guzmán**REQUISITO** Guías de Laboratorio N.º 1 y N.º 2**NOTA · Continúa desde las Guías N.º 1 y N.º 2**

Con el entorno ya instalado y capaz de ejecutar en Android, en esta práctica dejamos la configuración y empezamos a **programar**: entenderás cómo está organizado un proyecto Flutter y construirás tu primera pantalla componiendo widgets.

01 Objetivos

Objetivo general. Comprender la estructura de un proyecto Flutter y el modelo de composición de widgets, construyendo una primera interfaz de usuario y una navegación básica entre dos pantallas.

Objetivos específicos:

- Reconocer las carpetas y archivos clave de un proyecto Flutter y el rol de pubspec.yaml.
- Diferenciar `StatelessWidget` de `StatefulWidget` y entender el árbol de widgets.
- Construir una pantalla usando Scaffold, AppBar y widgets de layout (Column, Row, Container, Padding).
- Manejar estado local con `setState` para agregar interactividad.
- Navegar entre pantallas con `Navigator.push` y `Navigator.pop`.

02 Marco teórico

DEFINICIÓN · Todo es un widget

En Flutter, la interfaz se construye componiendo widgets: piezas que describen una parte de la pantalla (un texto, un botón, un espaciado, una fila). Los widgets se anidan formando un árbol de widgets, donde unos contienen a otros.

DEFINICIÓN · StatelessWidget vs StatefulWidget

StatelessWidget: widget inmutable; una vez construido no cambia (por ejemplo, un texto fijo o un icono).

StatefulWidget: widget que mantiene un estado interno que puede cambiar en el tiempo; al modificarlo con `setState`, Flutter vuelve a dibujar esa parte de la interfaz.

DEFINICIÓN · Widgets fundamentales de esta práctica

`MaterialApp` (raíz de la app), `Scaffold` (estructura básica de pantalla), `AppBar` (barra superior), `Text`, `Icon`, `Column` / `Row` (layout vertical/horizontal), `Container` y `Padding` (caja y márgenes internos).

03 Requisitos previos

Requisito indispensable. Haber completado las Guías N.º 1 y N.º 2 (entorno Flutter operativo y capaz de ejecutar en un dispositivo Android o en la web).

Conocimientos de apoyo:

Programación básica

Nociones de Dart

VS Code + Flutter

NOTA · Un primer vistazo a Dart

Dart usa clases (`class`), el método `build` devuelve el widget a mostrar, y la anotación `@override` indica que redefinimos un método heredado. No te preocupes por dominarlo hoy: lo iremos usando en cada práctica.

04 Desarrollo · Parte A — Anatomía del proyecto

1 Crear el proyecto y recorrer su estructura

1. Crea un proyecto nuevo (paleta de comandos `Ctrl + Shift + P` > **Flutter: New Project** > **Application**) con el nombre `mi_primera_ui`.
2. Observa en el explorador de VS Code la estructura generada. Las carpetas y archivos más importantes son:
 - `lib/` — código Dart de la app; aquí trabajarás casi siempre.
 - `lib/main.dart` — punto de entrada de la aplicación.
 - `pubspec.yaml` — configuración y dependencias del proyecto.
 - `android/` `ios/` `web/` — proyectos nativos por plataforma (se tocan poco al inicio).
 - `test/` — pruebas automatizadas.

2 Entender main.dart

El archivo generado arranca la app con `runApp`. En su forma esencial:

```
LIB/MAIN.DART · DART
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

NOTA · Qué hace cada línea

`import ... material.dart` trae los widgets de Material Design; `main()` es la función que se ejecuta al iniciar; `runApp` toma el widget raíz y lo dibuja en pantalla.

3 Revisar pubspec.yaml

1. Abre `pubspec.yaml`. Es el archivo donde se declaran el nombre, la versión y, sobre todo, las **dependencias** (paquetes externos) de la app.
2. La indentación en YAML es significativa: usa espacios, nunca tabulaciones. Tras editarlo, ejecuta `flutter pub get` para descargar los paquetes.

05 Desarrollo · Parte B — Tu primera interfaz con widgets

4 Crear el widget raíz de la app

1. Reemplaza el contenido de `lib/main.dart` por el siguiente widget raíz, un `StatelessWidget` que configura `MaterialApp`:

LIB/MAIN.DART · DART

```
import 'package:flutter/material.dart';

void main() => runApp(const MiApp());

class MiApp extends StatelessWidget {
  const MiApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Mi primera UI',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.teal,
        ),
        useMaterial3: true,
      ),
      home: const PantallaPerfil(),
    );
  }
}
```

5 Construir la pantalla con Scaffold y AppBar

1. Debajo del widget anterior, añade la pantalla de perfil. Usa `Scaffold` para la estructura, `AppBar` para la barra superior y `Column` para apilar elementos:

LIB/MAIN.DART · DART

```

class PantallaPerfil extends StatelessWidget {
  const PantallaPerfil({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Perfil')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const CircleAvatar(
              radius: 48,
              child: Icon(Icons.person, size: 48),
            ),
            const SizedBox(height: 16),
            const Text(
              'Ada Lovelace',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),
            const Text('Ingeniería Informática · UATF'),
          ],
        ),
      ),
    );
  }
}

```

2. Ejecuta con **F5**. Deberías ver una barra "Perfil" y, centrados, un avatar, un nombre y un subtítulo.

EJEMPLO · Anatomía del árbol de widgets

Scaffold → (AppBar + Center → Column → [CircleAvatar, SizedBox, Text, Text]). Cada widget es hijo del anterior: eso es el árbol de widgets en acción.

6 Ajustar el layout con Padding, Row e Icon

1. Dentro de la **Column**, debajo del último **Text**, agrega una fila con datos de contacto usando **Row**, **Icon** y **Padding**:

LIB/MAIN.DART · DART

```

const SizedBox(height: 24),
Padding(
  padding: const EdgeInsets.all(8.0),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.center,

```

```

    children: const [
      Icon(Icons.email, size: 18),
      SizedBox(width: 8),
      Text('ada@uatf.edu.bo'),
    ],
  ),
),

```

NOTA · Column vs Row

Column apila sus hijos en vertical; **Row** los coloca en horizontal. **SizedBox** inserta un espacio fijo y **Padding** agrega margen interno alrededor de un widget.

7 Agregar interactividad con un StatefulWidget

1. Para que algo cambie al tocar la pantalla necesitamos estado. Crea un **StatefulWidget** que cuente "me gusta":

LIB/MAIN.DART · DART

```

class BotonLikes extends StatefulWidget {
  const BotonLikes({super.key});

  @override
  State<BotonLikes> createState() => _BotonLikesState();
}

class _BotonLikesState extends State<BotonLikes> {
  int _likes = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Me gusta: $_likes'),
        IconButton(
          icon: const Icon(Icons.favorite),
          onPressed: () => setState(() => _likes++),
        ),
      ],
    );
  }
}

```

2. Agrega **const BotonLikes()** como último hijo de la **Column** de la pantalla de perfil (quita el **const** de la lista si el editor lo pide). Ejecuta y toca el corazón: el número sube.

EJEMPLO · El ciclo de setState

Al pulsar el botón, **setState** cambia **_likes** y le avisa a Flutter que reconstruya el widget. Flutter vuelve a ejecutar **build** y la pantalla muestra el nuevo valor.

06 Desarrollo · Parte C — Navegación entre pantallas

8 Crear una segunda pantalla

1. Añade una pantalla de detalle simple, que además sepa volver atrás:

```
LIB/MAIN.DART · DART
class PantallaDetalle extends StatelessWidget {
  const PantallaDetalle({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detalle')),
      body: Center(
        child: ElevatedButton(
          onPressed: () => Navigator.pop(context),
          child: const Text('Volver'),
        ),
      ),
    );
  }
}
```

9 Navegar con Navigator.push

1. En la pantalla de perfil, agrega un botón que abra la pantalla de detalle. Colócalo dentro de la `Column`:

```
LIB/MAIN.DART · DART
ElevatedButton(
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => const PantallaDetalle(),
      ),
    );
  },
  child: const Text('Ver detalle'),
),
```

DEFINICIÓN · La pila de navegación

`Navigator.push` apila una nueva pantalla encima de la actual; `Navigator.pop` la retira y regresa a la anterior. Es como una pila de tarjetas: la última en entrar es la primera en salir.

2. Ejecuta, pulsa **Ver detalle** para avanzar y **Volver** (o la flecha del AppBar) para regresar.

07 Verificación de resultados y entregable

Marca cada punto como verificación de que completaste la práctica:

- ☐ Sabes identificar `lib/main.dart` y `pubspec.yaml` y su función.
- ☐ La pantalla de perfil se muestra con AppBar, avatar, textos y una fila de contacto.
- ☐ El `StatefulWidget` de "me gusta" incrementa el contador al tocarlo.
- ☐ El botón "Ver detalle" navega a la segunda pantalla.
- ☐ El botón "Volver" (o la flecha del AppBar) regresa a la pantalla de perfil.

EJEMPLO · Qué entregar

El archivo `lib/main.dart` terminado y un documento (PDF o Word) con: (1) captura de la pantalla de perfil; (2) captura del contador de "me gusta" con un valor distinto de cero; (3) captura de la pantalla de detalle; y (4) **reto personal**: cambia el color del tema (`seedColor`), el nombre y agrega un widget más (por ejemplo, otro Icon o un Container con color de fondo) y descríbelo en 3–5 líneas.

08 Cuestionario

1. ¿Qué es el árbol de widgets y cómo se relaciona con el anidamiento del código?
2. Explica la diferencia entre `StatelessWidget` y `StatefulWidget` con un ejemplo de cada uno.
3. ¿Qué hace `setState` y por qué es necesario para actualizar la interfaz?
4. ¿Cuál es la diferencia entre `Column` y `Row`? ¿Para qué sirve `SizedBox`?
5. Describe qué ocurre en la pila de navegación al usar `Navigator.push` y luego `Navigator.pop`.

09 Rúbrica de evaluación

Criterio	Puntaje
Pantalla de perfil bien compuesta (layout y widgets)	30 %
Interactividad con <code>StatefulWidget</code> / <code>setState</code>	25 %
Navegación entre pantallas funcionando	20 %
Reto personal y capturas	10 %
Cuestionario resuelto	15 %

10 Referencias

- Flutter — Introduction to widgets. docs.flutter.dev/ui
- Flutter — Build a layout (tutorial). docs.flutter.dev/ui/layout/tutorial
- Flutter — Navigate to a new screen and back. docs.flutter.dev/cookbook/navigation/navigation-basics
- Flutter — Widget catalog. docs.flutter.dev/ui/widgets

Contenido basado en la documentación oficial de Flutter (v3.44). Guía elaborada para INF 662 — UATF.